



COURSE CODE/TITLE: CPE010

SECTION: CPE21S1

DATE PERFORMED: 30/07/26

DATE SUBMITTED: 01/08/2026

NAME: Duenas Ralph Florenz F.

INSTRUCTOR: Engr. Jimlord Quejado

ACTIVITY NO. ____ HOA 4.1

1. PROCEDURE OUTPUT

1. ILOA

```
#include <iostream>
#include <stack>

int main(){
    //create an object for the stack
    std::stack<int> newStack;

    //push to the stack
    newStack.push(3);
    newStack.push(8);
    newStack.push(15);

    std::cout<<"Stack Empty? "<<newStack.empty()<<std::endl;
    std::cout<<"The size of the stack: "<<newStack.size()<<std::endl;
    std::cout<<"The top of the element is: "<< newStack.top()<<std::endl;
    std::cout <<"Removing elements from the stack: \n";
    newStack.pop();
    std::cout<<"The top of the element is: "<< newStack.top()<<std::endl;
    newStack.pop();
    std::cout<<"The top of the element is: "<< newStack.top()<<std::endl;
    newStack.pop();
    std::cout<<"Stack Empty? "<<newStack.empty()<<std::endl;

    return 0;
}
```

```
Stack Empty? 0
The size of the stack: 3
The top of the element is: 15
Removing elements from the stack: The top element is: 8
The top element is: 3
Stack Empty?
```

2. ILOB

```

1  #include <iostream>
2
3  //static array, what do we need?
4  const size_t maxCap = 100;
5  int stack[maxCap];
6  int top = -1, i, newData;
7  //always -1 is the top of the stack if it is empty
8
9  //functions
10 void push();
11 void pop();
12 void Top();
13 bool isEmpty();
14 bool isFull();
15
16 int main() {
17     int choice; // Fixed: added missing semicolon
18     std::cout << "Enter the number of max elements for the new stack: ";
19     std::cin >> i;
20     push();
21
22     while(true) {
23         std::cout << "stack operations" << std::endl;
24         std::cout << "1. PUSH 2. POP 3. TOP 4. isEmpty" << std::endl;
25         std::cin >> choice;
26         switch(choice) {
27             case 1: push();
28                 break;
29             case 2: pop();
30                 break;
31             case 3: Top();
32                 break;
33             case 4:
34                 if(isEmpty()) std::cout << "Stack is empty" << std::endl;
35                 else std::cout << "Stack is not empty" << std::endl;
36                 break;
37         }
38     }
39
40     return 0;
41 }

```

```
49 bool isEmpty() {
50     //create a condition to check whether the top element is equal to -1
51     if (top == -1) return true;
52     return false;
53 }
54
55
56 bool isFull() {
57     //check if we reached the maximum capacity of the array
58     if (top >= (int)maxCap - 1) return true; // Fixed: prevent out of bounds
59     return false;
60 }
61
62
63 void push() {
64     //check if full -> if yes return an error
65     if (isFull()) {
66         std::cout << "stack overflow" << std::endl;
67         return;
68     }
69
70     std::cout << "New Value to add into the stack" << std::endl;
71     std::cin >> newData;
72
73     //increment the top(index by 1) then assign the new data;
74     stack[++top] = newData;
75 }
76
77
78 void Top() {
79     //check if the stack is empty
80     if (isEmpty()) {
81         std::cout << "stack underflow" << std::endl;
82         return;
83     }
84
85     std::cout << "The element on the top of the stack is " << stack[top] << std::endl;
86 }
87
88
89 void pop() {
90     //check if the stack is empty
91     if (isEmpty()) {
92         std::cout << "stack underflow" << std::endl;
93         return; // Fixed: added return to stop execution
94     }
95
96     //display the top value
97     std::cout << "popping: " << stack[top--] << std::endl; // Fixed: added semicolon & pop logic
98     top--;
99 }
100
101
102
103
104
```

```
Enter the number of max elements for the new stack: 67
New Value to add into the stack
78
stack operations
1. PUSH 2. POP 3. TOP 4. isEmpty
4
Stack is not empty
stack operations
1. PUSH 2. POP 3. TOP 4. isEmpty
```

3. MODIFICATION

```
1  #include <iostream>
2
3  //static array, what do we need?
4  const size_t maxCap = 100;
5  int stack[maxCap];
6  int top = -1, i, newData;
7  //always -1 is the top of the stack if it is empty
8
9  //functions
10 void push();
11 void pop();
12 void Top();
13 bool isEmpty();
14 bool isFull();
15 void display();
16
17 int main() {
18     int choice;
19     std::cout << "Enter the number of max elements for the new stack: ";
20     std::cin >> i;
21     push();
22
23     while(true) {
24         std::cout << "\nstack operations" << std::endl;
25         std::cout << "1. PUSH 2. POP 3. TOP 4. isEmpty 5. DISPLAY" << std::endl;
26         std::cout << "Enter choice: ";
27         std::cin >> choice;
28
29         switch(choice) {
30             case 1: push();
31                 break;
32             case 2: pop();
33                 break;
34             case 3: Top();
35                 break;
36             case 4:
37                 if(isEmpty()) std::cout << "Stack is empty" << std::endl;
38                 else std::cout << "Stack is not empty" << std::endl;
39                 break;
40             case 5: display();
41                 break;
42             default:
43                 std::cout << "Invalid choice! Please try again." << std::endl;
44                 break;
45         }
46     }
47
48     return 0;
49 }
```

```

51 bool isEmpty() {
52     //create a condition to check whether the top element is equal to -1
53     if (top == -1) return true;
54     return false;
55 }
56
57 bool isFull() {
58     //check if we reached the maximum capacity of the array
59     if (top >= (int)maxCap - 1) return true;
60     return false;
61 }
62
63 void push() {
64     //check if full -> if yes return an error
65     if (isFull()) {
66         std::cout << "stack overflow" << std::endl;
67         return;
68     }
69
70     std::cout << "New Value to add into the stack: ";
71     std::cin >> newData;
72
73     //increment the top(index by 1) then assign the new data;
74     stack[++top] = newData;
75 }
76
77 void Top() {
78     //check if the stack is empty
79     if (isEmpty()) {
80         std::cout << "stack underflow" << std::endl;
81         return;
82     }
83
84     std::cout << "The element on the top of the stack is " << stack[top] << std::endl;
85 }
86
87 void pop() {
88     //check if the stack is empty
89     if (isEmpty()) {
90         std::cout << "stack underflow" << std::endl;
91         return;
92     }
93
94     //display the top value and decrement top
95     std::cout << "popping: " << stack[top--] << std::endl;
96 }
97
98 // Function to display all elements in the stack
99 void display() {
100     if (isEmpty()) {
101         std::cout << "Stack is empty, nothing to display." << std::endl;
102         return;
103     }
104
105     std::cout << "Elements in stack (top to bottom): ";
106     for (int j = top; j >= 0; j--) {
107         std::cout << stack[j] << " ";
108     }
109     std::cout << std::endl;
110 }

```

```

Enter the number of max elements for the new stack: 3
New Value to add into the stack: 1

stack operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. DISPLAY
Enter choice: 3
The element on the top of the stack is 1

stack operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. DISPLAY
Enter choice: 1
New Value to add into the stack: 2

stack operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. DISPLAY
Enter choice: 3
The element on the top of the stack is 2

stack operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. DISPLAY
Enter choice: 1
New Value to add into the stack: 3

stack operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. DISPLAY
Enter choice: 5
Elements in stack (top to bottom): 3 2 1

stack operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. DISPLAY
Enter choice: 2
popping: 3

stack operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. DISPLAY
Enter choice: 5
Elements in stack (top to bottom): 2 1

stack operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. DISPLAY
Enter choice: 

```

- OBSERVATIONS

This seatwork shows how a simple stack works using an array, where data can only be added or removed from one end. Because the array size is set to 100, the stack has a fixed limit, meaning it cannot take more numbers once full. The code uses the Last-In, First-Out rule, so the last number put in is always the first one taken out. To keep the program safe from crashing, functions like `isFull()` and `isEmpty()` make sure the code never adds numbers to a full array or removes numbers from an empty one. Finally, the `display()` function uses a simple loop to print every number from top to bottom without deleting anything from the stack.

- ANALYSIS OF THE PROCEDURE

In the main procedure (`main.cpp`), the program initialized a fixed-size integer array to represent a static stack with a maximum capacity of 100 elements and tracked the top element using an index initialized to -1. The program prompted the user to specify the maximum number of elements and performed an initial `push()` operation to insert the first value into the stack. It then entered an interactive control loop offering menu options for stack management. Selecting option 1 triggered `push()`, which verified capacity via `isFull()` before incrementing the top index and storing a new integer value. Option 2 called `pop()`, which checked for underflow using `isEmpty()` before displaying and removing the top element by decrementing the top index. Option 3 utilized `Top()` to inspect and display the current top element without removing it, while Option 4 evaluated `isEmpty()` to report whether the stack contained any elements. Lastly, Option 5 executed `display()`, which iterated backward from the current top index down to 0 to print all elements in top-to-bottom order without modifying the stack structure.

- 3. ILOB.2

```

1 ~ #include <iostream>
2 ~ #include "stackList.h"
3
4 ~ int main() {
5
6     std::cout << "--- PUSHING 'C' 'P' 'E' ONTO THE STACK ---- \n"
7
8     // push
9     push<char>('C');
10    push<char>('P');
11    push<char>('E');
12
13    // Display the current top
14    top<char>(); // Top should be 'E'
15
16    std::cout << "\n -----Display all element ----- \n";
17
18    displayAll<char>();
19
20    std::cout << "\n ----- Popping all the elements ----- \n";
21
22 ~ while (!isEmpty<char>()) {
23     std::cout << " Popped: " << pop<char>() << std::endl;
24 }
25
26    std::cout << "\n ----- Testing the Underflow ----- \n";
27    pop<char>();
28
29    return 0;
30 }

```

```
1 ~ #ifndef STACK_LIST_H
2 ~ #define STACK_LIST_H
3
4 ~ // class node that contains data and the next
5 ~ template <typename T>
6 ~ class Node {
7 ~ public:
8 ~     T data;      // VALUE THAT WE NEED STORE
9 ~     Node* next;  // POINTER TO THE NEXT NODE
10 ~ };
11
12 ~ template <typename T>
13 ~ Node<T>* head = nullptr;
14
15 ~ // isEmpty function (placed before pop/top/display to avoid forward declaration errors)
16 ~ template <typename T>
17 ~ bool isEmpty() {
18 ~     return head<T> == nullptr;
19 ~ }
20
21 ~ // push
22 ~ template <typename T>
23 ~ void push(T newData) {
24 ~     // create a new node and assign the newData
25 ~     Node<T>* newNode = new Node<T>;
26 ~     newNode->data = newData;
27
28 ~     // new node points to current top, then becomes the new top
29 ~     newNode->next = head<T>;
30 ~     head<T> = newNode;
31 ~ }
```

```

33 // pop
34 template <typename T>
35 T pop() {
36     // checks if the stack is empty
37     if (head<T> == nullptr) {
38         std::cout << "Stack underflow." << std::endl;
39         return T();
40     }
41
42     // save the current top node
43     Node<T>* temp = head<T>;
44     // store its data
45     T tempVal = temp->data;
46
47     // move the head to the next node
48     head<T> = head<T>->next;
49     // free the removed node
50     delete temp;
51     // return the removed value
52     return tempVal;
53 }
54
55 // top
56 template <typename T>
57 void top() {
58     // error catching:
59     if (isEmpty<T>()) {
60         std::cout << "Stack underflow" << std::endl;
61         return;
62     }
63
64     // display the Top Node
65     std::cout << "Top of stack: " << head<T>->data << std::endl;
66 }
67
68 // Display all
69 template <typename T>
70 void displayAll() {
71     if (isEmpty<T>()) {
72         std::cout << "Stack is empty" << std::endl;
73         return;
74     }
75
76     std::cout << "Stack contents (top to bottom): ";
77     Node<T>* temp = head<T>;
78     while (temp != nullptr) {
79         std::cout << "\n| " << temp->data << " | ";
80         temp = temp->next;
81     }
82     std::cout << std::endl;
83 }
84
85 #endif

```

```
--- PUSHING 'C' 'P' 'E' ONTO THE STACK ---
```

```
Top of stack: E
```

```
-----Display all element -----
```

```
Stack contents (top to bottom):
```

```
| E |  
| P |  
| C |
```

```
----- Popping all the elements -----
```

```
Popped: E
```

```
Popped: P
```

```
Popped: C
```

```
----- Testing the Underflow -----
```

```
Stack underflow.
```

```
PS C:\Users\TIPQC\Documents\Duenas CPE010\ilob2> █
```

- ANALYSIS OF THE CODE IN B.2

In this HOA the (main.cpp), the program started by bringing in the code from `stackList.h`. It printed a starting message and used `push<char>()` three times to add the letters "C", "P", and "E" into the stack. Next, it is called `top<char>()` to check the item at the top of the stack ("E") without removing it, making sure first that the stack was not empty. It then ran `displayAll<char>()` to print all the letters currently in the stack from top to bottom by following the linked nodes starting from `head`. After that, it entered a `while` loop using `!isEmpty<char>()` to repeatedly call `pop<char>()`, which printed each letter and deleted its node from memory until the stack was empty. Finally, it called `pop<char>()` one last time on the empty stack to test and show the "Stack underflow" error message before ending the program with return 0;.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

- Main

```

1  #include <iostream>
2  #include <string>
3  #include "stackList.h"
4
5  bool isMatchingPair(char open, char close) {
6      if (open == '(' && close == ')') return true;
7      if (open == '{' && close == '}') return true;
8      if (open == '[' && close == ']') return true;
9      return false;
10 }
11
12 // Algorithm to check if the string has balanced symbols
13 bool isBalanced(const std::string& expr) {
14     // 1. Process character by character
15     for (char ch : expr) {
16         if (ch == '(' || ch == '{' || ch == '[') {
17             push<char>(ch);
18         }
19         else if (ch == ')' || ch == '}' || ch == ']') {
20
21             if (isEmpty<char>()) {
22                 return false;
23             }
24
25             char topChar = pop<char>();
26
27             if (!isMatchingPair(topChar, ch)) {
28
29                 while (!isEmpty<char>()) { pop<char>(); }
30                 return false;
31             }

```

```
32 |  
33 | }  
34 | bool valid = isEmpty<char>();  
35 |  
36 | while (!isEmpty<char>()) {  
37 | |   pop<char>();  
38 | | }  
39 |  
40 |   return valid;  
41 | }  
42 |  
43 | int main() {  
44 |   // Test expressions from the activity sheet  
45 |   std::string expressions[] = {  
46 |       "(A+B)+(C-D)",  
47 |       "((A+B)+(C-D))",  
48 |       "((A+B)+[C-D])",  
49 |       "((A+B)+[C-D])"  
50 |   };  
51 |  
52 |   std::cout << "--- BALANCED SYMBOLS CHECKER ---\n\n";  
53 |  
54 |   for (const std::string& expr : expressions) {  
55 |       std::cout << "Expression: " << expr << "\n";  
56 |  
57 |       if (isBalanced(expr)) {  
58 |           std::cout << "Result: VALID (Y)\n";  
59 |       } else {  
60 |           std::cout << "Result: INVALID (N)\n";  
61 |       }  
62 |  
63 |       std::cout << "\n";  
64 |   }  
65 |  
66 |   return 0;  
67 | }
```

- Stacklist

```
1  #ifndef STACK_LIST_H
2  #define STACK_LIST_H
3
4  #include <iostream>
5
6  // Node structure for the linked list
7  template <typename T>
8  struct Node {
9      T data;
10     Node<T>* next;
11 };
12
13 // Global head pointer for the stack
14 template <typename T>
15 Node<T>* head = nullptr;
16
17 // Check if the stack is empty
18 template <typename T>
19 bool isEmpty() {
20     return head<T> == nullptr;
21 }
22
23 // Push an item onto the stack
24 template <typename T>
25 void push(T newData) {
26     Node<T>* newNode = new Node<T>;
27     newNode->data = newData;
28     newNode->next = head<T>;
29     head<T> = newNode;
30 }
31
32 // Pop an item from the stack and return its value
33 template <typename T>
34 T pop() {
35     if (isEmpty<T>()) {
36         std::cout << "Stack underflow.\n";
37         return T();
38     }
39     Node<T>* temp = head<T>;
40     T tempVal = temp->data;
41     head<T> = head<T>->next;
42     delete temp;
43     return tempVal;
44 }
45
```

```

46 // Get the top item without removing it
47 template <typename T>
48 void top() {
49     if (isEmpty<T>()) {
50         std::cout << "Stack underflow\n";
51         return;
52     }
53     std::cout << "Top of stack: " << head<T>-->data << "\n";
54 }
55
56 // Display all items in the stack from top to bottom
57 template <typename T>
58 void displayAll() {
59     if (isEmpty<T>()) {
60         std::cout << "Stack is empty\n";
61         return;
62     }
63     std::cout << "Stack contents (top to bottom): ";
64     Node<T>* temp = head<T>;
65     while (temp != nullptr) {
66         std::cout << "\n| " << temp->data << " |";
67         temp = temp->next;
68     }
69     std::cout << "\n";
70 }
71
72 #endif

```

- OUTPUT

Expression: (A+B)+(C-D)

Result: VALID (Y)

Expression: ((A+B)+(C-D)

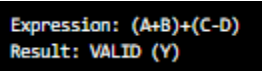
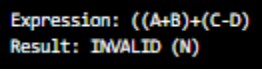
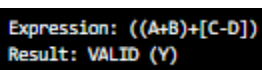
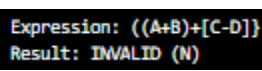
Result: INVALID (N)

Expression: ((A+B)+[C-D])

Result: VALID (Y)

Expression: ((A+B)+[C-D])}

Result: INVALID (N)

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
$(A+B)+(C-D)$	Y		All opening parentheses (have matching closing) in the correct order.
$((A+B)+(C-D)$	N		Missing a closing) at the end. The stack is not empty when processing finishes.
$((A+B)+[C-D])$	Y		All parenthetical () and square bracket [] pairs are correctly balanced and nested.
$((A+B)+[C-D])\}$	N		Mismatched closing delimiter: opened with (but closed with }

Summary of lessons learned

- In this HOA, I learned how to implement and use a stack data structure using a linked list (stackList.h). From my first main.cpp code, I learned basic stack operations by pushing characters like 'C', 'P', and 'E' onto the stack, using top() to check the newest item, and using a loop with pop() to remove elements until triggering an underflow test. I then applied these exact same operations—push(), pop(), and isEmpty()—to solve a real problem: checking if math expressions have balanced brackets. Opening symbols like (, {, and [are pushed onto the stack, and when a closing symbol like), }, or] is found, the top item is popped to verify if they match. Using a linked list for the stack allows it to grow dynamically with nodes and pointers without fixed size limits.

Analysis of the supplementary activity

- Based on my understanding, the program reads an expression character by character. When it encounters an opening bracket, it pushes it onto the stack using dynamic node creation in stackList.h. When it finds a closing bracket, it pops the top symbol from the stack to see if they form a matching pair. If the brackets do not match or if the stack is empty when a closing symbol appears, the program marks the expression as invalid. Finally, if the stack is completely empty after reading the whole expression, it confirms that all symbols are properly balanced.

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

- I think I did very well in this HOA. I was able to successfully implement the algorithm using a linked list stack and correctly determine which expressions were valid or invalid. However, I still need more practice in handling template functions cleanly and managing dynamic pointers without causing memory leaks when errors happen mid-way through execution.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Copilot

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code assistance, explanation of concept and the flow of the code itself.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: I used Copilot to elaborate the thoughts I have in my mind without replacing the original thought and also for getting the formulas that are needed. I affirm that my use of AI complies with TIP's AI usage policy.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.